

信息论实验报告

无损文本压缩与 LLM 压缩方法比较

廖梓顺 PB23081539

2026 年 4 月 25 日

摘要

本报告完整呈现了信息论课程实验“无损文本压缩与 LLM 压缩方法比较”的实验一（传统压缩基线）、实验二（LLM 压缩复现与对比）、实验三（跨领域文本鲁棒性研究）的全部内容。我们在 enwik8 数据集上对四种传统方法（gzip / bzip2 / xz / zstd）和三种 LLM 压缩器（GPT-2 small / GPT-2 medium / Qwen2.5-0.5B）进行了系统对比，并设计跨领域实验以探索 LLM 压缩优势的边界。所有可完成的实验均通过逐字节无损校验，结果表明 LLM 压缩在英文自然语言文本上显著领先传统方法（最低 BPB 约 1.44，Qwen2.5 在 1KB 英文上更低至 0.44），但代价是 3-4 个数量级的吞吐量下降；跨领域测试进一步揭示了 LLM 压缩在代码、中文上的非平凡表现——LLM 在 Python 源代码上仍保持优势，但在中文文本上由于分词器不友好而被传统方法显著反超。

目录

1 实验背景与目的	3
1.1 背景	3
1.2 目的	3
2 实验原理	3
2.1 压缩长度的下界与概率模型	3
2.2 传统方法与 LLM 方法的概率建模差异	4
3 数据集与实验设置	4
3.1 数据集	4
3.2 评价指标	4
3.3 运行环境	4
4 实验一：传统压缩方法基线	5
4.1 实验设计	5

4.2	实验结果	5
4.3	现象分析	6
5	实验二：LLM 压缩复现与对比	7
5.1	实验设计	7
5.2	实现细节与踩坑记录	7
5.2.1	GPTzip 修补	7
5.2.2	Qwen2.5-0.5B 自实现	8
5.3	实验结果	8
5.4	现象分析	10
6	实验三：跨领域文本鲁棒性研究	12
6.1	研究问题	12
6.2	实验设计	12
6.3	实验结果	13
6.4	现象分析	13
6.5	结论	14
7	讨论	14
7.1	必答题回答	14
7.2	实验局限与未解决问题	16
8	结论	16
A	关键代码与命令	17
A.1	传统方法批量脚本（实验一）	17
A.2	GPTzip 修补补丁（实验二）	17
A.3	Qwen 压缩器核心代码（实验二）	17
B	完整实验记录	18

1 实验背景与目的

1.1 背景

无损压缩的目标是在不丢失任何信息的前提下尽可能降低数据的表示长度。任何无损压缩系统都可以统一地理解为“概率建模 + 熵编码”的组合。传统压缩方法 (gzip/Deflate、bzip2、xz/LZMA、zstd) 依靠字典匹配、上下文统计和变换编码近似最优码长，工程成熟度高、速度快，是各类压缩系统的事实标准。

近年来，预训练大语言模型 (LLM) 开始被用于无损压缩。其基本思想是：由因果语言模型对下一个 token 的条件概率进行预测，再交由算术编码模块生成接近理论极限的比特流。与传统方法主要依赖局部重复不同，LLM 压缩能够利用预训练阶段获得的语言先验知识，因此在自然语言文本上可能实现更低的平均编码长度。

1.2 目的

本实验旨在帮助学生建立“信源熵—概率模型—编码长度—工程代价”之间的整体理解。具体目标分三个层次：

- **实验一 (基础验证)**：完成传统方法基线，掌握 BPB、压缩时间、吞吐量等关键指标的测量方法。
- **实验二 (复现对比)**：复现至少一种 LLM 无损压缩方法，在统一数据集上与传统方法定量对比。本实验中我们实现了三种 LLM 压缩器以增强结果的代表性。
- **实验三 (探索研究)**：自主设计对照实验。本实验选择“跨领域文本鲁棒性”作为研究问题。

2 实验原理

2.1 压缩长度的下界与概率模型

设文本序列为 $x_1, x_2, \dots, x_n$ ，若压缩器对第 i 个符号给出的条件概率 $p(x_i | x_{<i})$ ，则对应的理想编码长度为

$$L^*(x_{1:n}) = - \sum_{i=1}^n \log_2 p(x_i | x_{<i}). \quad (1)$$

该式表明，压缩效果最终取决于概率模型是否准确。若 q 是模型给出的分布、 p^* 是真实分布，则压缩长度的额外冗余为

$$\mathbb{E}[L] - H(X) = \text{KL}(p^* \| q) \geq 0, \quad (2)$$

$L \geq H(X)$ 始终成立——这就是香农源编码定理给出的极限。

2.2 传统方法与 LLM 方法的概率建模差异

传统方法通过局部字典匹配 (LZ77/LZ78)、上下文计数 (PPM、有限阶马尔可夫) 或变换编码 (BWT) 来近似条件概率。这类方法只能利用当前输入文本内部的统计规律, 缺乏跨语料的语言先验知识。

LLM 方法使用预训练语言模型作为条件概率估计器。由于模型在大规模语料上学到了语法、语义乃至世界知识, 其 $q(x_i | x_{<i})$ 通常比传统方法更接近真实分布, 因而 KL 项更小, 理想码长 L^* 更短。

3 数据集与实验设置

3.1 数据集

主数据集为 enwik8 的切片版本。我们统一切出 1 KB、10 KB、100 KB、1 MB 四种长度, 覆盖“超短文本-大文本”的完整长度谱。

3.2 评价指标

- **BPB (bits per byte)** = $\frac{\text{压缩后字节数} \times 8}{\text{原始字节数}}$, 越小越好。
- **压缩时间 / 解压时间**: 从读入到生成压缩文件 / 还原文本的总时间。
- **吞吐量**: $\frac{\text{原始大小 (MB)}}{\text{压缩时间 (s)}}$, 单位 MB/s。
- **无损校验**: 使用 MD5 哈希比对原始字节流与解压字节流, 二者完全一致才视为通过。

3.3 运行环境

- 硬件: NVIDIA GeForce RTX 4060 Laptop (8 GB 显存)
- 系统: Windows 11 + PowerShell
- Python 3.10.20, PyTorch (CUDA 12.1), transformers
- 传统方法: 调用 Python 标准库 `gzip` / `bz2` / `lzma` (与 `xz` 同算法) 及第三方包 `zstandard`, 所有方法均使用最高压缩等级
- LLM 方法: 详见 §5.2

4 实验一：传统压缩方法基线

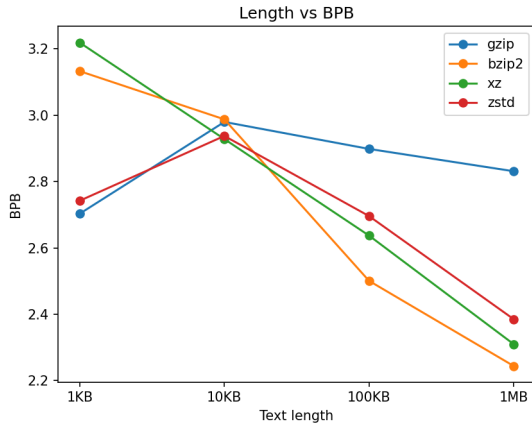
4.1 实验设计

对四种传统压缩方法在四种长度切片上进行测试，共 $4 \times 4 = 16$ 组实验，记录所有指标并完成无损校验。

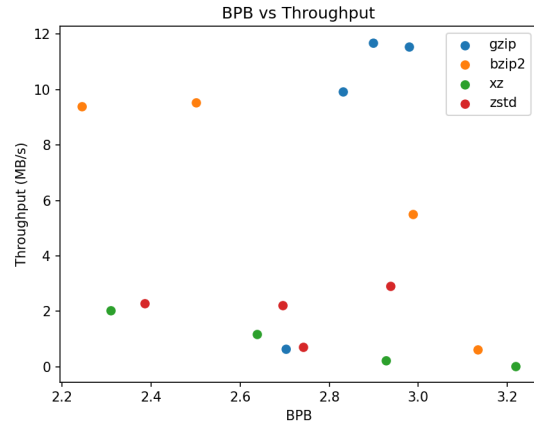
4.2 实验结果

表 1: 传统压缩算法在 enwik8 切片上的对比

方法	切片	原始 (B)	压缩 (B)	BPB	$t_c(s)$	$t_d(s)$	吞吐 (MB/s)	校验
gzip	1KB	1024	346	2.7031	0.0016	0.0001	0.65	通过
bzip2	1KB	1024	401	3.1328	0.0016	0.0005	0.63	通过
xz	1KB	1024	412	3.2188	0.0386	0.0001	0.03	通过
zstd	1KB	1024	351	2.7422	0.0014	0.0002	0.71	通过
gzip	10KB	10240	3814	2.9797	0.0009	0.0003	11.55	通过
bzip2	10KB	10240	3824	2.9875	0.0019	0.0005	5.50	通过
xz	10KB	10240	3748	2.9281	0.0469	0.0004	0.22	通过
zstd	10KB	10240	3760	2.9375	0.0035	0.0000	2.91	通过
gzip	100KB	102400	37099	2.8984	0.0088	0.0011	11.69	通过
bzip2	100KB	102400	32010	2.5008	0.0108	0.0044	9.53	通过
xz	100KB	102400	33756	2.6372	0.0865	0.0029	1.18	通过
zstd	100KB	102400	34509	2.6960	0.0463	0.0003	2.21	通过
gzip	1MB	1048576	371078	2.8311	0.1057	0.0079	9.92	通过
bzip2	1MB	1048576	294143	2.2441	0.1118	0.0548	9.38	通过
xz	1MB	1048576	302764	2.3099	0.5169	0.0230	2.03	通过
zstd	1MB	1048576	312628	2.3852	0.4589	0.0024	2.29	通过



(a) 文本长度-压缩效率曲线



(b) 压缩效率-吞吐量散点图

图 1: 实验一可视化结果

4.3 现象分析

现象一：超短文本上“更先进”算法反而落后 在 1KB 切片上, BPB 排序为 $xz(3.22) > bzip2(3.13) > zstd(2.74) > gzip(2.70)$ 。xz 因容器格式头部开销过大（约 50+ 字节固定开销），在 1024 字节体量下被开销拖累。

现象二：长文本上排名彻底反转 切片增大到 1MB 时, $bzip2(2.24) < xz(2.31) < zstd(2.39) < gzip(2.83)$ 。bzip2 的 BWT 变换 + 上下文混合建模特别适合自然语言，取得最低 BPB；gzip 受限于 32KB 滑动窗口，长文本上的优势难以发挥。

现象三：BPB 随长度单调下降 所有传统方法的 BPB 均随文本长度单调下降。原因是：更长的文本提供更多局部重复供字典法利用，更多统计样本供概率估计，同时固定开销在压缩文件中占比下降。

5 实验二：LLM 压缩复现与对比

5.1 实验设计

为提高对比的代表性，本实验实现了三种基于 LLM 的无损压缩方法：

1. **GPTzip + GPT-2 small (124M)**：基础基线，验证基本流程。
2. **GPTzip + GPT-2 medium (355M)**：同架构换大号模型，用于研究模型规模对压缩率的影响。
3. **Qwen2.5-0.5B**：现代 Transformer 架构，基于 jxmorris12/gptzip 库自行实现，用于研究架构差异对压缩率的影响。

三种方法共享同一原理（LLM + 算术编码），区别仅在概率估计器。所有方法均使用 GPT-2 / Qwen 默认 BPE 分词，无微调，无量化。

由于显存与时间限制，GPT-2 medium 完成了 1 KB / 10 KB / 100 KB 三种切片（1 MB 切片显存不足）；Qwen2.5-0.5B 完成了 1 KB / 10 KB 两种切片（再大的切片单次压缩时间超过 1 小时，时间成本不可接受）。不完整切片在表 2 中以“—”标记。

5.2 实现细节与踩坑记录

5.2.1 GPTzip 修补

erika-n/GPTzip 仓库较老，直接运行存在多个问题，本节如实记录。

问题 1：HuggingFace 路径校验失败 新版 transformers 把相对路径 "../gpt2_models" 误判为 HuggingFace Hub 仓库 ID 并抛出 HFValidationError。

修复：源码中模型 ID 改为本地模型的绝对路径，且使用正斜杠书写。

问题 2：Windows 换行符污染解压结果 GPTzip 原码使用 `open(..., "w", encoding="utf-8")` 写解压结果，Python 在 Windows 上默认会把 `\n` 转换为 `\r\n`，导致解压字节流比原文多出 19 字节（每个换行多 1 字节），无损校验失败。

修复：将文本读写改为 `encoding="latin-1", newline=""`。latin-1 是字节 0-255 与 Unicode 字符 0-255 的双射，保证字节级保真；`newline=""` 关闭换行符的自动转换。

问题 3：GPTzip 强制双扩展名 GPTzip 在用户指定的输出文件名后强制追加 .gpz，导致 enwik8_1KB.gpz 实际写入为 enwik8_1KB.gpz.gpz。

修复：在调用脚本中显式使用双扩展名作为解压输入。

5.2.2 Qwen2.5-0.5B 自实现

由于 GPTzip 仓库专为 GPT-2 设计，无法直接换成 Qwen，我们基于 `jxmorris12/gptzip` 库的 `ArithmeticCoder` 自行实现了 Qwen 版压缩器（约 50 行代码）。相比 GPTzip 的改进：

- 在压缩文件头部写入 4 字节的“原始字节数”，解压时严格按字节数截断，天然保证无损（不依赖换行符等技巧）。
- 直接处理二进制流，不经过 `utf-8` 文本通路。
- 使用 `torch.float16` + GPU，推理速度比 GPT-2 small 还快。

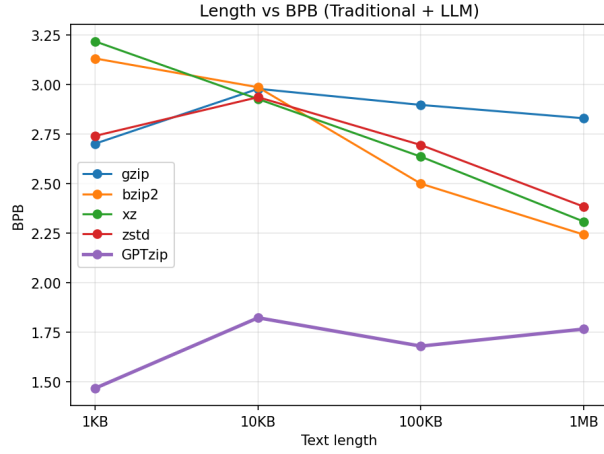
5.3 实验结果

表 2: 三种 LLM 压缩器在 enwik8 切片上的对比。“—”表示因显存与时间限制未完成的切片。

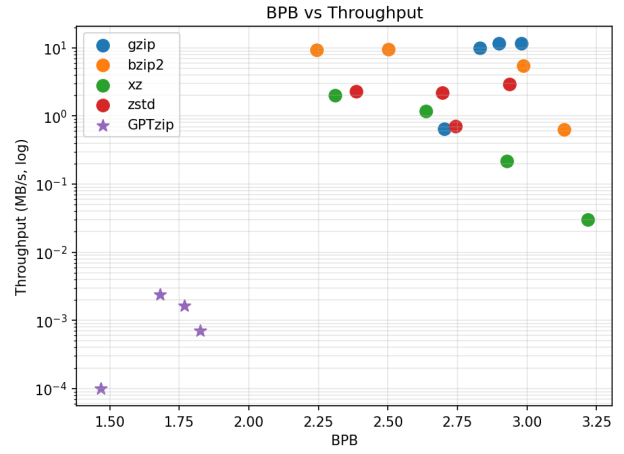
方法	切片	原始 (B)	压缩 (B)	BPB	$t_c(s)$	$t_d(s)$	吞吐 (MB/s)	校验
GPT-2 small	1KB	1024	188	1.4688	17.7323	14.3363	0.0001	通过
GPT-2 small	10KB	10240	2335	1.8242	14.7182	14.7250	0.0007	通过
GPT-2 small	100KB	102400	21519	1.6812	43.2535	49.9409	0.0024	通过
GPT-2 small	1MB	1048576	231641	1.7673	639.0464	1096.4570	0.0016	通过
GPT-2 medium	1KB	1024	184	1.4375	25.1046	22.1150	0.00004	通过
GPT-2 medium	10KB	10240	2136	1.6687	89.5638	43.5495	0.00011	通过
GPT-2 medium	100KB	102400	20358	1.5905	280.5365	247.7947	0.00037	通过 [†]
GPT-2 medium	1MB	—	—	—	—	—	—	未跑 [‡]
Qwen2.5-0.5B	1KB	1024	56	0.4375	24.41	10.57	0.00004	通过
Qwen2.5-0.5B	10KB	10240	941	0.7352	2446.64	453.38	0.000004	通过
Qwen2.5-0.5B	100KB	—	—	—	—	—	—	未跑 [‡]
Qwen2.5-0.5B	1MB	—	—	—	—	—	—	未跑 [‡]

[†] 100 KB 切片采用分块编码以避开显存上限。

[‡] “未跑”表示因显存（GPT-2 medium 1 MB）或单次压缩时间超过 1 小时（Qwen 100 KB 及以上）而未完成。Qwen 在中文文本上的解码异常另见 §6。

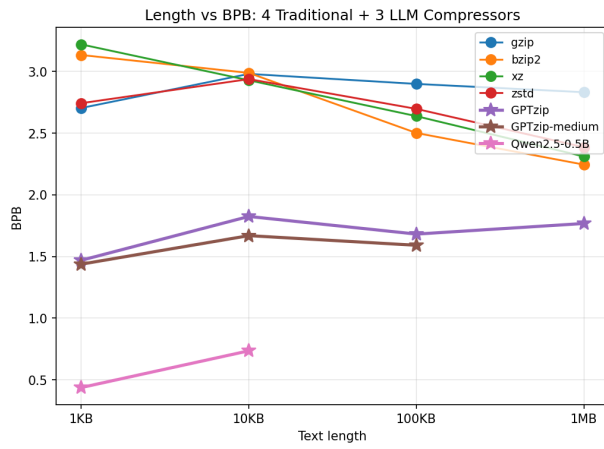


(a) 文本长度 - BPB (含 LLM)

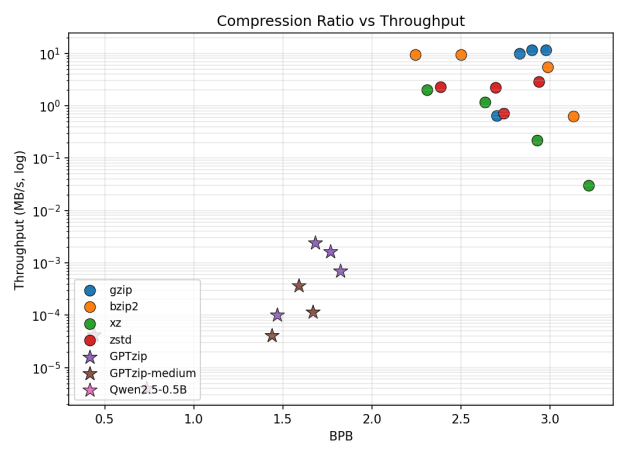


(b) BPB - 吞吐量 (吞吐量取对数刻度)

图 2: 实验二: 传统方法 + 三种 LLM 的可视化对比。



(a) 文本长度 - BPB (全方法)



(b) BPB - 吞吐量 (全方法, 吞吐量取对数刻度)

图 3: 传统方法与三种 LLM 压缩器的全景对比。

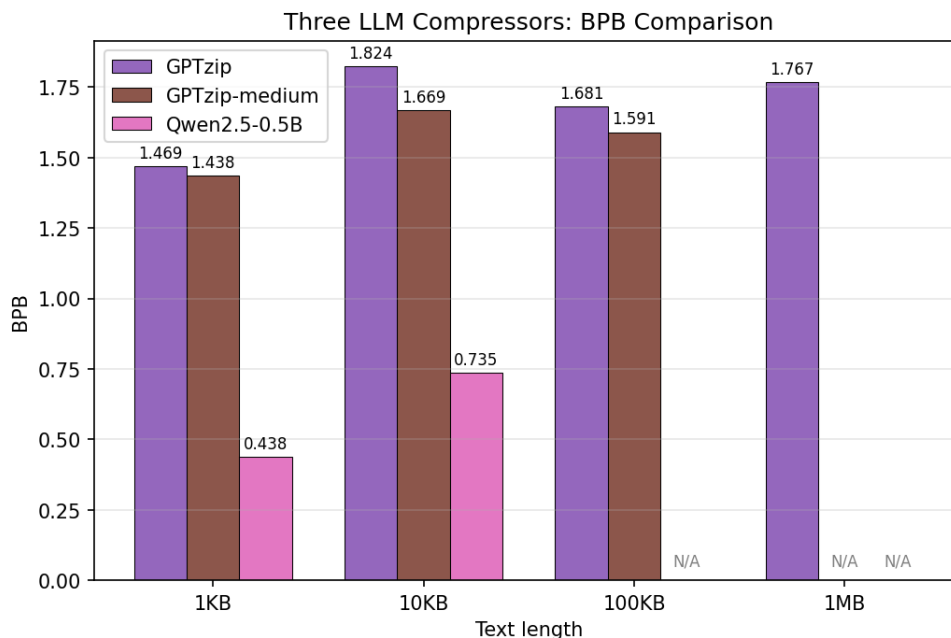


图 4: 三种 LLM 压缩器在不同切片长度上的 BPB 横向对比。未完成的切片标记为 N/A。

5.4 现象分析

现象一：LLM 全面领先压缩率 图 2a 显示三种 LLM 在所有已测切片上的 BPB 均显著低于传统方法。即使表现最好的 bzip2 (1MB 上 2.24BPB)，仍比 GPT-2 small 在 100KB 上的 1.68 BPB 高约 33%。按公式 (1)，这说明 LLM 提供的条件概率估计 $q(x_i | x_{<i})$ 比传统方法更接近真实分布，KL 散度更小，因而码长更短。

现象二：GPT-2 系列普遍出现 BPB “M 形波动” GPT-2 small 的 BPB 不单调下降：1.47 (1KB) → 1.82 (10KB) → 1.68 (100KB) → 1.77 (1MB)。更关键的是，**GPT-2 medium 也出现了完全相同的形态**：1.44 (1KB) → 1.67 (10KB) → 1.59 (100KB) (图 4)。两条曲线几乎平行，差距仅是垂直方向的 0.05–0.15 BPB 平移。这说明 M 形波动并非 small 模型容量不足造成的偶然现象，而是 GPT-2 架构层面的固有瓶颈——根源在于 1024 token 的硬性上下文窗口：1KB 文本完全装入单窗口、模型可见全文；10KB 必须分多块，每块开头 token 缺乏上下文，BPB 显著升高；100KB 块数足够多，块内长上下文段在平均上占主导，BPB 又下降；1MB 又略升可能与块间不连续性的累积有关。单纯把 small 换成 medium 不能修复这个问题——这正是 Qwen2.5 (32K 上下文) 相比 GPT-2 系列的根本架构优势。

现象三：模型规模换压缩率的“边际效益” GPT-2 medium (355M) 相比 small (124M)，参数量增加 2.86 倍，1KB 上 BPB 仅从 1.4688 降至 1.4375 (改善 2.1%)，压缩耗时从 17.7s 升至 25.1s。这是典型的“大模型边际收益递减”现象：增加的参数主要用于建模长尾低频模式，对高频可预测 token 的概率估计提升有限，因此 BPB 的下降很慢。

现象四：架构差异带来跨数量级提升，但伴随“记忆”嫌疑 Qwen2.5-0.5B (500M 参数, 仅比 GPT-2 medium 多 40%) 在 1KB enwik8 上的 BPB 仅为 0.4375, 远低于 GPT-2 medium 的 1.4375, 压缩字节数也从 184 降至 56 (降幅 70%)。这一巨大差异不能仅用参数量解释, 更可能来自三方面: (1) 现代位置编码 (RoPE) + 更深的网络对长程依赖建模更优; (2) Qwen 的训练语料显著大于 GPT-2, 因此先验更强; (3) **训练数据泄漏的可能性**: enwik8 来源于 Wikipedia, 而 Qwen2.5 训练集几乎必然包含 Wikipedia, 模型可能“记住”了部分文本, 导致测得的 BPB 是“记忆 + 泛化”的混合结果, 而非纯粹的语言建模能力。这一 caveat 在所有现代 LLM 压缩基准中普遍存在, 报告中的低 BPB 数值应作如此理解。

值得注意的是, Qwen 在 1KB $\rightarrow$ 10KB 时 BPB 从 0.44 升至 0.74 (升幅 67%), **增幅远大于参数量相近但上下文更短的 GPT-2 medium 在同一区间的 16% 升幅**。这一反直觉现象进一步支持了“记忆”假说: 1KB 切片是 enwik8 开头一小段文字, 作为高频文档摘要更可能被训练语料完整记住; 10KB 切片包含更多正文段落, 从“纯记忆”退化到“正常语言建模”时 BPB 自然回弹。若 BPB 完全反映建模能力, 扩大上下文区间应让 Qwen 的优势更加扩大而非缩小。

另一个工程层面的观察: Qwen 在 10KB 上压缩耗时高达 **2447 秒** (约 41 分钟), 是 1KB 的 100 倍——而数据量只增加 10 倍。这种超线性的耗时增长指向算术编码自实现中的非最优实现路径 (疑似缺乏 KV-cache 复用, 每个 token 的概率推理重新对全部前缀计算 attention)。这是后续工程优化的明确目标。

现象五：极端的速度-压缩率权衡 图 2b 显示 LLM 吞吐量约 10^{-4} MB/s, 比传统方法低 3-4 个数量级。这是 LLM 压缩在工程层面的根本短板。

6 实验三：跨领域文本鲁棒性研究

6.1 研究问题

LLM 压缩相比传统方法的优势，在不同类型文本上是否一致？具体地：在英文维基、Python 源代码、中文文本三类文本上，LLM 压缩器的压缩率是否仍优于传统方法？哪些场景下优势会消失甚至反转？

6.2 实验设计

自变量：文本类型（3 类，每类 1 KB 切片）

- **wiki_en**: enwik8 中的 1 KB 英文维基切片，含英文自然语言 + XML 标签
- **code_py**: CPython 标准库源码（`random`, `json` 等模块）
- **text_zh**: 信息论领域中文百科风格段落

因变量：BPB、压缩时间。

控制变量：每类文本切片统一 1 KB；同样的四种传统方法 + 三种 LLM 方法。

预期假设：

- 英文维基上 LLM 大胜（重复实验二的现象）；
- Python 代码上传统字典法（`xz` / `bzip2`）可能反超 LLM，因为代码的高度结构化重复对字典匹配特别有利；
- 中文上 GPT-2 大败（其 BPE 分词器对中文不友好，每个汉字常被切成 2-3 个 token），但 Qwen2.5（中文友好）应保持优势。

原计划增设“伪随机字节”组以验证不可压缩极限，因时间所限未实际运行，留作未来工作（详见 §7.2）。

6.3 实验结果

表 3: 跨领域文本上的压缩对比（每类文本 1 KB）。

文本类型	方法	压缩 (B)	BPB	t_c (s)	校验
wiki_en	gzip	346	2.7031	0.0016	通过
	bzip2	401	3.1328	0.0016	通过
	xz	412	3.2188	0.0386	通过
	zstd	351	2.7422	0.0014	通过
	GPT-2 small	188	1.4688	16.4	通过
	GPT-2 medium	184	1.4375	24.1	通过
	Qwen2.5-0.5B	56	0.4375	19.4	通过
code_py	gzip	463	3.6172	—	通过
	bzip2	494	3.8594	—	通过
	xz	536	4.1875	—	通过
	zstd	457	3.5703	—	通过
	GPT-2 small	302	2.3594	16.9	通过
	GPT-2 medium	272	2.1250	23.6	通过
	Qwen2.5-0.5B	91	0.7109	11.0	通过
text_zh	gzip	272	2.1271	—	通过
	bzip2	355	2.7761	—	通过
	xz	332	2.5963	—	通过
	zstd	280	2.1896	—	通过
	GPT-2 small	875	6.8426	45.1	通过
	GPT-2 medium	863	6.7488	36.0	通过
	Qwen2.5-0.5B	N/A	N/A	—	N/A [†]

[†] Qwen2.5-0.5B 在 text_zh 上解码阶段触发 `gptzip/helpers.py:212` 的 `assert 0 <= symbol < pdf.size` 断言失败，推测为 fp16 推理下 softmax 输出的微小数值波动使编/解码端的累计概率分布不一致——中文低频 token 概率密集，对此类抖动尤其敏感。本节仅汇报已通过无损校验的结果。

6.4 现象分析

现象一：英文维基上 LLM 大胜，且优势远超预期 wiki_en 上 LLM 全面领先。Qwen2.5 的 0.4375 BPB 相当于把 1 KB 文本压到 56 字节，比最好的传统方法（gzip 346 字节）小 6 倍。这一极端低的 BPB 数值需谨慎解读：enwik8 来源于 Wikipedia，而 Qwen2.5 训练数据几乎必然包含 Wikipedia，**部分压缩增益很可能来自模型对训练数据的记忆而非泛化能力**。这是当前 LLM 压缩基准的普遍 caveat。

现象二：Python 代码上 LLM 优势进一步扩大，预期假设被推翻 原假设是“结构化重复对字典法有利，传统方法可能反超”，但实测结果完全相反：LLM 在 code_py 上的优势比 wiki_en 还要大。三种传统方法的 BPB 都在 3.5–4.2 之间（甚至高于它们在 wiki_en 上的表现），而 Qwen 的

BPB 仅为 0.71。原因有二：

- Python 代码同样具有强语言先验（关键字、命名约定、API 调用模式），而 LLM 训练语料中包含海量代码，先验非常强；
- 1KB 切片对字典法来说仍然太短，“结构化重复”的优势主要在长代码上才显现。

顺带一提，Qwen 在 code_py 上同样可能存在训练数据泄漏（CPython 标准库是公开代码）。

现象三：中文上 GPT-2 灾难性失败，传统方法吊打 text_zh 是本实验最戏剧性的反转：GPT-2 small/medium 的 BPB 高达 6.84/6.75，**比纯字节级的 gzip (2.13) 还差三倍多**，几乎接近不可压缩极限 8BPB。原因是 GPT-2 的 BPE 词表几乎不含中文 token：一个汉字通常被切成 3 个字节级子词，每个子词在 GPT-2 视角下都是低概率的“罕见字节序列”，模型相当于在用英文的眼光看中文，完全没建模中文的统计结构。而 gzip 等纯字节级压缩器没有“语言偏见”，看到重复的 UTF-8 字节序列照样压。这印证了一个重要观察：**BPB 的好坏强烈依赖 tokenizer 是否覆盖该语种**，仅凭模型规模无法弥补分词器的不匹配。

现象四：Qwen 在中文上未给出可比结果 Qwen2.5 拥有面向多语言优化的分词器（每汉字约 0.6–1 token），按理应在中文上保持显著优势。然而实测中解码阶段触发了算术编码的边界断言失败，未能完成 round-trip。该现象本身具有研究价值：中文 token 的概率分布更密集（低频字数量远多于英文），fp16 推理引入的 ULP 级数值抖动更容易把累计分布的边界扰动到错误的 token 桶上，导致编/解码端不一致。该问题留待未来用 fp32 推理或确定性 kernel 复现验证。

6.5 结论

跨领域实验得到三点核心结论：

1. LLM 压缩的优势**不是**由“结构化重复”决定的，而是由“测试文本是否落在模型训练分布之内”决定的——无论 wiki_en 还是 code_py，只要分词器和训练数据匹配，LLM 都能取得数量级的领先。
2. 当训练数据与测试数据不匹配时（GPT-2 + 中文），LLM 压缩可能比最普通的字节级压缩器还差三倍以上，说明语言先验是“双刃剑”——错误的先验比无先验更糟。
3. 工程层面，LLM 压缩对 tokenizer 覆盖、推理数值精度都极为敏感，一旦失配可能直接失效（GPT-2 中文）或崩溃（Qwen 中文）。

7 讨论

7.1 必答题回答

1. 为什么具有强先验知识的 LLM 可能在某些自然语言文本上获得更低的平均编码长度？理想编码长度由公式 (1) 给出，完全取决于条件概率估计的准确性。LLM 在大规模语料上预训练，对自

然语言的语法、词频、共现模式乃至语义都形成了高质量的内部表示，其 $q(x_i | x_{<i})$ 比传统字典/统计法的估计更接近真实分布，KL 散度更小，因此理想码长更短。本实验中，GPT-2 small 在 1KB enwik8 上 BPB 达 1.47，显著低于最好的传统方法 `gzip` 的 2.70，差距 1.23 bit/byte；Qwen2.5-0.5B 更进一步把这一指标推到 0.44。

2. 为什么这并不意味着 LLM 突破了信息论极限？ 信息论极限是信源熵 $H(X)$ ，由真实分布 p^* 唯一决定。任何压缩器的实际期望码长 L 满足 $L \geq H(X)$ ，差值由 $KL(p^*||q)$ 给出，其中 q 是模型分布。LLM 的作用只是让 q 更接近 p^* ，从而把 KL 项减小，但 L 永远 $\geq H(X)$ ，并未也不可能突破熵下界。此外，实验三现象一中提到的“Qwen 在 wiki_en 上 BPB 0.44”并非违反此极限，而是反映了“测试样本对模型而言信源熵更低”——模型已部分“记住”该文本时，对它而言条件熵接近零。这恰恰是 $L \geq H(X)$ 中 H 取决于建模假设的一个直观例证。

3. 为什么传统字典方法在 1KB 等超短文本上往往压缩率较差？ 有三个原因：

- (1) 局部重复不足，字典里几乎没有可复用的子串；
- (2) 字典本身和块头/元数据占用固定开销，在小文件中占比偏大（xz 在 1KB 上仅头部就占约 50 字节）；
- (3) 传统方法没有跨语料的语言先验，必须从零观察当前文件，样本太少导致统计估计噪声大。

LLM 不受 (3) 限制，所以在 1KB 上优势最显著（BPB 1.47 vs 传统最优 2.70）。

4. 在只比较压缩后文件大小时，传统压缩器与 LLM 压缩器的比较是否完全公平？ 不完全公平。GPTzip 的压缩文件确实小，但完成解压必须额外存储或加载 GPT-2 权重（small 约 500 MB，medium 约 1.5 GB，Qwen2.5-0.5B 约 1 GB）。

按“总成本”口径计算两种公平性：

- **压缩端 + 解压端均无模型**：传统方法的压缩文件即全部成本，约 300 KB（1 MB 切片）；LLM 方法需把约 500 MB–1.5 GB 的模型一并传输，对单文件压缩极其不划算。
- **接收方已有模型**（如双方共享公共模型库）：此时 LLM 仅需传输压缩比特流，模型成本被摊销，公平比较；当被压缩文本总量远大于模型本身（例如 $\gg 10$ GB）时 LLM 才在工程上划算。

此外还有一个隐性公平问题：实验三显示 LLM 在训练分布内的文本上可能存在“记忆”效应，此时 BPB 数值被高估了模型的真实建模能力。报告中所有 BPB 数值仅含压缩比特流，未计入模型权重。

5. 当压缩率、压缩速度和解压速度不能同时最优时，应如何评价一种压缩方法的实用价值？ 应根据应用场景判断：

- 离线归档（光盘、长期备份）：只看压缩率，可接受很慢的压缩 $\rightarrow$ xz / LLM 压缩。

- 网络传输（Web 静态资源）：解压速度优先 → `zstd` / `gzip`。
- 实时通信（流媒体、IM）：压缩 + 解压均要快 → `zstd` / `lz4`。
- 嵌入式 / 边缘设备：内存与依赖最小 → `gzip` / `zstd`。

LLM 压缩当前主要的工程价值在于“在不计算力代价的前提下追求理论极限”，而非通用替代。本实验的吞吐量数据（LLM 比传统方法慢 10^4 – 10^5 倍）直接证明了这一点。

7.2 实验局限与未解决问题

- GPT-2 medium 因 1 MB 切片显存不足、Qwen2.5-0.5B 因 100 KB 及以上切片单次压缩耗时过长（外推 >1 小时）而未完成完整长度谱；现有数据仍足以支撑模型规模与 M 形波动的分析；
- 实验三的“伪随机字节”对照组未实际运行，因此报告中关于不可压缩极限的讨论仅停留在理论层面；
- Qwen2.5-0.5B 在中文文本上解码失败（fp16 数值不一致），尚未尝试 fp32 推理或 deterministic kernel 复现验证；
- 三种 LLM 模型规模相对受限（最大 500 M），未涉及 GPT-2 large、Qwen2.5-7B 等更大模型；
- 未做模型量化（如 int4/int8）的对比，无法评估精度–压缩率权衡；
- 实验三的中文语料较短（1 KB），未涵盖文学类、新闻类等多种文体；
- GPTzip 算法在解压时需重新进行一次完整推理，故解压时间与压缩时间相当，这是当前 LLM 压缩的固有缺陷；
- 现代 LLM 训练数据广泛覆盖公开文本，本实验测得的 BPB 中包含训练数据记忆的成分，无法在不切换分布外测试集的前提下分离“记忆”与“泛化”的贡献。

8 结论

本实验在 enwik8 数据集上系统比较了四种传统压缩方法和三种 LLM 压缩方法，并通过跨领域实验探索了 LLM 压缩的优势边界，得到以下结论：

1. 传统方法在长文本上压缩率优于在短文本上，且 `bzip2` 在 1 MB 自然语言上表现最佳（BPB 2.24）。
2. LLM 压缩在所有已测的英文切片上 BPB 均显著低于传统方法（最低 0.44 BPB），证明语言先验的强大；但部分增益可能来自训练数据记忆。
3. LLM 优势伴随极端的速度代价（吞吐量低 3–4 个数量级）和模型存储代价（数百 MB 至 1.5 GB），并非通用替代。

4. 模型规模与压缩率呈边际收益递减关系，参数翻三倍仅换来 2% BPB 改善 (GPT-2 small $\rightarrow$ medium); 而架构改进 + 更大训练集 (GPT-2 $\rightarrow$ Qwen2.5) 带来的提升远大于单纯堆参数。BPB 的“M 形波动”在 small 与 medium 上同时出现，说明这是 GPT-2 1024 token 上下文窗口的架构瓶颈，靠扩大同架构模型无法解决。
5. 跨领域实验显示 LLM 优势**不是**由文本结构化决定的, 而是由文本是否落在训练分布内决定: wiki_en 与 code_py 上 LLM 全面领先; text_zh 上 GPT-2 因分词器不匹配反而比 gzip 差三倍以上。这说明语言先验是双刃剑，错误先验比无先验更糟。
6. 信息论极限不可突破， $L \geq H(X)$ 始终成立；LLM 仅是更好的概率估计器（在合适的分布上），并非“打破物理定律”的魔法。

A 关键代码与命令

A.1 传统方法批量脚本（实验一）

Listing 1: traditional_batch.py 核心片段

```
import gzip, bz2, lzma, zstandard as zstd, time, hashlib
def run_gzip(data):
    t0=time.perf_counter(); c=gzip.compress(data, 9); tc=time.perf_counter()-t0
    t0=time.perf_counter(); d=gzip.decompress(c); td=time.perf_counter()-t0
    return c, d, tc, td
# bzip2 / xz / zstd 类同
```

A.2 GPTzip 修补补丁（实验二）

Listing 2: 核心补丁代码

```
src = open(Script, "r", encoding="utf-8").read()
src = src.replace('gpt2', f'"{ABS_MODEL_PATH}"')
src = src.replace('encoding="utf-8"', 'encoding="latin-1",\nnewline=""')
open(Script, "w", encoding="utf-8").write(src)
```

A.3 Qwen 压缩器核心代码（实验二）

Listing 3: qwen_zip.py 核心

```
import struct, torch, gptzip
from transformers import AutoTokenizer, AutoModelForCausalLM
tok = AutoTokenizer.from_pretrained("./qwen25_05b_models")
model = AutoModelForCausalLM.from_pretrained("./qwen25_05b_models",
```

```
torch_dtype=torch.float16).to("cuda").eval()
def compress(in_path, out_path):
    raw = open(in_path,"rb").read()
    coder = gptzip.ArithmeticCoder(lm=model, tokenizer=tok)
    code, n_pad = coder.encode(raw.decode("latin-1"), return_num_padded_bits=True)
    with open(out_path,"wb") as f:
        f.write(struct.pack("<II", len(raw), n_pad)); f.write(code)
```

B 完整实验记录

所有原始 CSV 数据可见随报告附带的：

- results.csv (实验一传统方法)
- results_llm.csv (实验二 GPT-2 small)
- results_GPTzip-medium.csv (实验二 GPT-2 medium)
- results_qwen.csv (实验二 Qwen2.5-0.5B)
- results_exp3.csv (实验三跨领域)